

DDAGMA Upload Portal — System Documentation

Application: General Movements Assessment (GMA) Video Upload Portal
Short name: DDAGMA Upload
Publisher: Marcus Fan in collaboration with the Developmental Disabilities Association
Audience: Clinic administrators, developers, and maintainers
Document date: August 4, 2026

1. Purpose and overview

DDAGMA is a secure web application that lets clinic staff issue **single-use temporary portal links** to parents. Parents upload one GMA assessment video (with the date the video was recorded). The file is stored in the clinic’s connected **Microsoft OneDrive** account with a standardized filename. After a successful upload, staff receive an **email notification** that includes the child’s name and a direct OneDrive link to the file.

The product has two primary surfaces:

Surface	URL	Users
Parent upload portal	/ (entered via /portalaccess/{token})	Parents / caregivers
Admin console	/setup	Allowlisted Microsoft accounts
Public info	/info, /privacy, /tos	Anyone

Technology stack

- **Frontend / API:** Next.js (App Router) + TypeScript + React + ShadCN UI
 - **Storage of links & settings:** Upstash Redis
 - **File destination:** Microsoft Graph / OneDrive
 - **Auth:** Microsoft Entra ID (MSAL) with PKCE; signed HttpOnly cookies for portal/admin sessions
 - **Email:** Azure Communication Services
 - **Optional on Vercel:** Vercel Blob for durable OneDrive token cache
-

2. Operator instructions (day-to-day use)

2.1 First-time clinic setup

1. Deploy the app and configure environment variables (see §6).
2. Ensure your Microsoft Entra app registration allows the accounts you need (typically organizational + personal via /common) and lists redirect URIs:
 - {APP_URL}/api/auth/onedrive/callback
 - {APP_URL}/api/auth/upload-access/callback
3. Seed at least one admin email via ALLOWED_ADMIN_EMAILS and/or Settings → Allowed admin emails.
4. Open /setup and sign in with an allowlisted Microsoft account.
5. Connect the **receiving OneDrive** account (must also be on the allowlist).
6. In Settings, set:
 - Upload folder name (default uploads)
 - Reference Excel workbook name and child-name / EDC columns
 - Link buffer time and expiry
 - Admin allowlist

7. Confirm the reference workbook exists in that OneDrive and the child list loads.

2.2 Generating a parent link

1. In `/setup` , pick a child from the reference sheet (or enter name + EDC as supported by the UI).
2. Generate a link. The app stores child name + EDC on the Redis link record.
3. Copy the full URL (`{origin}/portalaccess/{token}`) and send it to the parent through your normal clinic channel.
4. Links start in a short **provisioning** (buffer) period, then become usable until they expire or are used once.

2.3 Parent upload flow

1. Parent opens the portal link.
2. If still in the buffer window, they see a countdown (`/link-expired?reason=pending...`).
3. When active, they land on `/` with a portal session cookie.
4. They pick **Date recorded**, then **Add video** (mobile) or drag/drop (desktop).
5. Upload runs; progress is shown.
6. On success: file is in OneDrive, staff are emailed, the link is marked **used**, and the parent cookie is cleared.

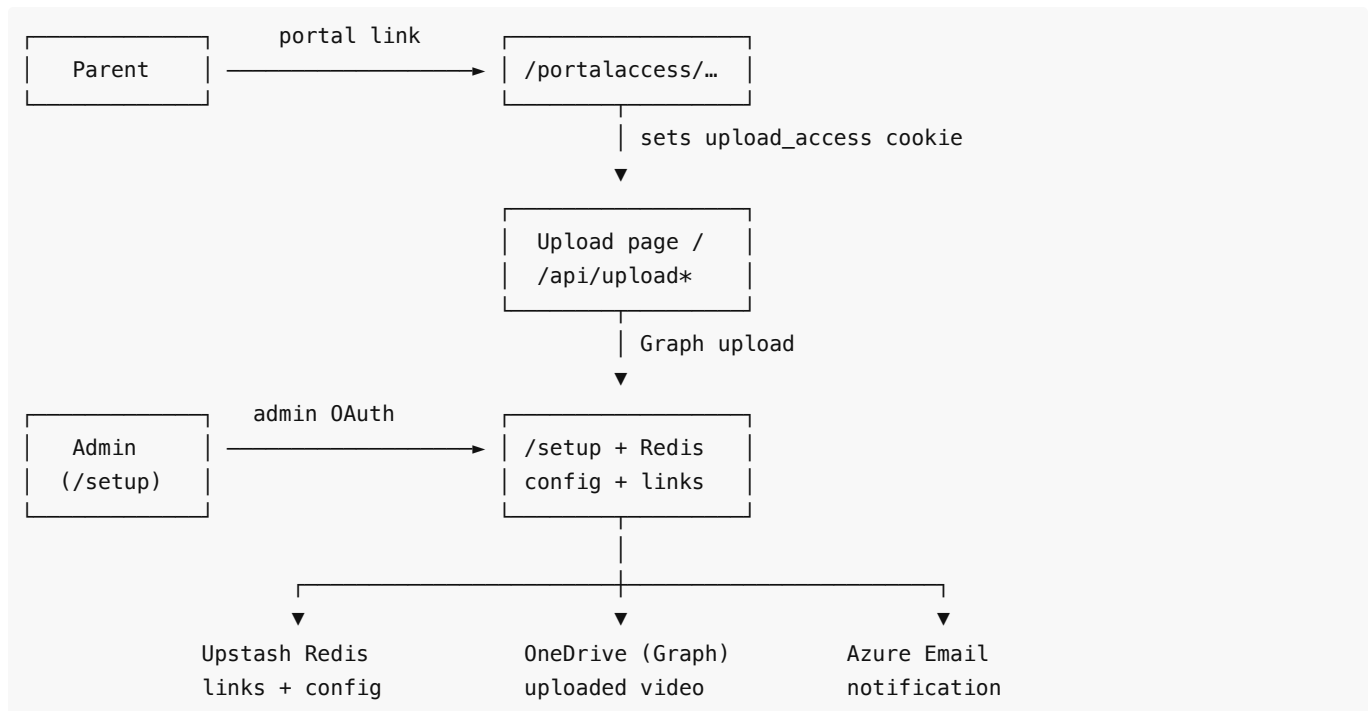
2.4 After upload

- Check email for subject: `{ChildName}'s parent has uploaded a new video to your OneDrive` .
- Open the OneDrive link in the message, or browse the configured folder.
- In `/setup` , dismiss used links when no longer needed.

2.5 Changing the receiving OneDrive

Use **Change receiving OneDrive** in the admin console. Only allowlisted accounts can become the destination. Reconnect after secret/client ID changes or if Graph tokens are revoked.

3. Architecture summary



Key server module: `lib/server.ts` — Redis, config, links, MSAL/token cache, cookies, Graph helpers, OAuth callback orchestration.

Client upload orchestration: `lib/upload.ts` .

Email: `lib/email.ts` .

Shared defaults: `lib/appConfig.ts` .

4. Temporary links and Redis lifecycle

4.1 Storage

Redis key	Contents	TTL
link: {uuid}	createdAt, childName, edc, expiresAt, optional usedAt	Set to link expiry at creation; cleared TTL behavior after consume (kept until dismissed)
app:config	Partial config overrides	None

4.2 States (admin UI)

State	Meaning
provisioning	$\text{now} < \text{createdAt} + \text{bufferTimeMs}$ (default 60 seconds). Parent cannot upload yet.
pending	Buffer elapsed; waiting for a successful upload.
used	usedAt set after <code>/api/upload/complete</code> .

4.3 Parent-facing checks (`/portalaccess/{token}`)

Result	Behavior
Missing / expired / used	Redirect to <code>/link-expired</code>
Still in buffer	Redirect to pending countdown with <code>availableAt</code>
Active	Set portal cookie and redirect to <code>/</code>

Important: Opening the link does **not** consume it. Consumption happens only after a successful upload finalize.

4.4 Filename derivation

At upload time the server reads the portal link and builds:

```
GMA Video {Child Name} {DD.MM.YYYY}_{ageWeeks}{extension}
```

- **Child name** and **EDC** come from the link.
- **Date recorded** is supplied by the parent.
- **Age (weeks)** = whole weeks from EDC → date recorded (`lib/age.ts`).
- Calculation is **server-side** before naming; the parent UI does not preview age.

5. API routing reference

All API routes under `app/api/**` use `dynamic = "force-dynamic"` . Auth is enforced inside handlers (there is no `Next.js middleware.ts`).

5.1 Upload APIs

POST /api/upload

- **Auth:** Portal access (`canAccessUploadPortal`) — valid portal cookie with usable link, or valid admin cookie.
- **Body:** `multipart/form-data`
 - `file` — video file
 - `dateRecorded` — `YYYY-MM-DD`
- **Server steps:**
 1. Resolve filename from portal link + date recorded.
 2. Obtain OneDrive access token.
 3. PUT file bytes to Graph (`.../root:/{folder}/{filename}:/content`).
- **Success (200):** `{ id, name, webUrl, size }`
- **Errors:** `401` access required; `400` missing fields; `500` Graph/config failures.
- **Use when:** File size $\leq$ `maxSimpleFileSizeBytes` (default **4 MB**).

POST /api/upload/session

- **Auth:** Same portal access gate.
- **Body (JSON):** `{ filename, fileSize, dateRecorded }`
- **Server steps:** Validate size, resolve final filename, create Graph upload session.
- **Success (200):** `{ uploadUrl, expirationDateTime, filename, uploadChunkSizeBytes, ... }`
- **Client then:** Uploads chunks with `PUT` **directly to Microsoft** `uploadUrl` using `Content-Range` headers (default chunk **10 MB**).
- **Use when:** File larger than the simple-upload threshold (up to `maxFileSizeBytes` , default **4 GB**).

POST /api/upload/complete

- **Auth:** Same portal access gate.
- **Body (JSON):** `{ webUrl?, name? }` — OneDrive item metadata from the upload result.
- **Server steps:**
 1. If portal token + `webUrl` present $\rightarrow$ send notification email (child name from link; recipients = connected OneDrive username + allowlisted admins).
 2. Consume link (`usedAt`).
 3. Clear portal `upload_access` cookie.
- **Success (200):** `{ ok: true }`
- Email failures are **logged** and do **not** fail the response (file is already stored).
- Admin sessions without a portal token: no consume / no cookie clear.

GET /api/upload/context

- **Auth:** Portal access gate.
- **Success (200):** `{ childName, edc, availableAt, expiresAt, fromLink }`
- Used by `FileProvider` on `/` to prefill child metadata and show the upload window in instructions.

5.2 End-to-end client upload sequence

Implemented in `lib/upload.ts` and triggered from `components/uploadArea.tsx` .

1. Parent selects video + date recorded
2. If size $\leq$ simple limit:
 - `POST /api/upload` (XHR + progress) $\rightarrow$ OneDrive result
- Else:
 - `POST /api/upload/session`
 - `PUT` chunks $\rightarrow$ Microsoft `uploadUrl` $\rightarrow$ OneDrive result

3. POST /api/upload/complete { webUrl, name }
4. UI shows success; link is single-use thereafter

Mobile Safari notes: Mobile uses a native full-size transparent `<input type="file">` (not `react-dropzone open()`). Empty MIME types are accepted when the file looks like video. Do not clear `input.value` in the same turn as reading the `File`.

5.3 Config and link management APIs

GET /api/config

- **Auth:** None (public).
- **Returns:** Full merged `AppConfig` (limits, folder name, admin emails, etc.).

PUT /api/config

- **Auth:** Valid admin cookie.
- **Body:** Partial config patch.
- **Returns:** Merged config after Redis update.

DELETE /api/config

- **Auth:** Admin.
- **Effect:** Clears Redis overrides; defaults + env merge apply again.

POST /api/generate-link

- **Auth:** Admin.
- **Body:** { childName, edc }
- **Returns:** { token, url, createdAt, childName, edc, expiresInSeconds }

GET /api/links

- Lists stored links with public URLs.
- **Note for maintainers:** Currently has **no admin auth check** — treat as a hardening candidate before wide public deployment.

DELETE /api/links

- **Body:** { token } — dismisses/removes a link.
- **Note:** Also currently **unauthenticated** — same hardening note as GET.

5.4 OneDrive browse APIs (admin)

GET /api/onedrive/browse

- **Auth:** Admin.
- **Returns:** { folders, workbooks } from the connected drive (for Settings pickers).

GET /api/onedrive/child-names

- **Auth:** Admin.
- **Returns:** Children parsed from the reference Excel workbook (name , edc , column metadata).

5.5 Authentication APIs

Method	Path	Purpose
GET	/api/auth/admin/login	Start admin Microsoft login (PKCE, flow=admin)
GET	/api/auth/upload-access/login	Start upload-access OAuth (optional loginHint)
GET	/api/auth/upload-access/callback	OAuth callback (shared handler)

GET	/api/auth/onedrive/login	Connect/replace receiving OneDrive (requires admin; clears prior tokens)
GET	/api/auth/onedrive/callback	OAuth callback for setup flow
GET	/api/auth/onedrive/status	Connection status + redirect URI diagnostics
DELETE	/api/auth/onedrive/status	Disconnect receiving OneDrive (admin; keeps admin session)

5.6 Non-API route handler

GET /portalaccess/[token]

Activates a parent session when the link is usable (see §4.3). Sets the signed portal cookie and redirects to `/`.

6. Authentication and cookies

6.1 Cookies

Cookie	Purpose	Notes
onedrive_pkce	PKCE code verifier	HttpOnly, short-lived (~10 min)
onedrive_auth_flow	setup admin upload-access	HttpOnly, short-lived
upload_access	Signed portal or admin session	HttpOnly; Secure in production; ~14 days

Cookie HMAC secret resolution: `UPLOAD_ACCESS_SECRET` → else `UPSTASH_REDIS_REST_TOKEN` → else a **dev-only** fallback (do not rely on fallback in production).

6.2 Portal vs admin payload

- **Portal:** { type: "portal", token, exp }
- **Admin:** { type: "admin", username, exp } — username must remain allowlisted

6.3 canAccessUploadPortal

1. If OneDrive is not connected → allowed (open gate; Graph calls will fail later).
2. Valid admin cookie → allowed.
3. Portal cookie + Redis link exists and not used → allowed.
4. Else → denied (may include upload-access login URL).

6.4 Admin console gate

`app/setup/layout.tsx` requires `hasValidAdminAccess`. Unauthenticated users are sent through `/api/auth/admin/login`.

7. Configuration (AppConfig)

Defaults live in `lib/appConfig.ts`. Overrides persist in Redis via Settings / `PUT /api/config`.

Field	Default	Role
folderName	uploads	OneDrive destination folder under drive root
bufferTimeMs	60 000 (1 min)	Delay before a new link becomes usable

linkExpiryTimeMs	3 600 000 (1 h)	Link lifetime / Redis TTL at create
fileDetails.maxFileCount	1	One video per upload
fileDetails.maxFileSizeBytes	4 GB	Hard max
fileDetails.maxSimpleFileSizeBytes	4 MB	Threshold for /api/upload vs session
fileDetails.uploadChunkSizeBytes	10 MB	Resumable chunk size
referenceSheetName	REFERENCE.xlsx	Workbook for child picker
childNameColumn / edcColumn	Child Name / EDC	Excel headers (or letters)
allowedAdminEmails	(defaults + env)	Who may use /setup and own the drive

Accepted upload types default to `video/*` (client also accepts empty MIME + common video extensions for iOS).

8. Environment variables

Variable	Required	Purpose
NEXT_PUBLIC_AZURE_CLIENT_ID	Yes	Entra application (client) ID
AZURE_CLIENT_SECRET	Yes (prod / silent refresh)	Confidential client secret
NEXT_PUBLIC_APP_URL / APP_URL	Yes	Canonical origin for links and redirects
UPSTASH_REDIS_REST_URL	Yes	Redis REST endpoint
UPSTASH_REDIS_REST_TOKEN	Yes	Redis token (also may sign cookies)
UPLOAD_ACCESS_SECRET	Recommended in prod	Dedicated cookie signing secret
ALLOWED_ADMIN_EMAILS	Recommended	Comma/newline list of admin emails
AZURE_AUTHORITY	Optional	Default https://login.microsoftonline.com/common
ONEDRIVE_REDIRECT_URI	Optional	Narrow registered redirect URI
BLOB_READ_WRITE_TOKEN / BLOB_STORE_ID	Vercel	Persist MSAL OneDrive token cache
ONEDRIVE_CACHE_PATH	Optional local	Override local token cache path
AZURE_COMMUNICATION_SERVICES_CONNECTION_STRING	For email	ACS connection string (Communication Services Keys blade)
AZURE_EMAIL_SENDER_ADDRESS	For email	Verified MailFrom on a domain linked to that ACS resource

Email setup reminder: Connection string comes from the Communication Services resource → Settings → Keys. The Email Communication Services resource has no keys; its domain must be **connected** under Communication Services → Email → Domains. Mismatched sender domains produce `DomainNotLinked`.

Local LAN testing: `next.config.ts` may list `allowedDevOrigins` (e.g. a machine LAN IP) so phones can load HMR assets during `next dev`.

9. Email notifications

Module: `lib/email.ts`

Trigger: `/api/upload/complete` after a portal upload with `webUrl`.

Item	Behavior
Subject	<code>{childName}'s parent has uploaded a new video to your OneDrive</code>
Body	Plain + HTML with OneDrive open link (<code>webUrl</code>) and optional file name
Recipients	Connected OneDrive account username + <code>allowedAdminEmails</code> (deduplicated)
Failure mode	Logged; upload completion still succeeds

If ACS env vars are missing, send is skipped with a server warning.

10. Local development and deployment

10.1 Local

```
npm install
# Create .env.local with Azure, Upstash, and optional ACS variables
npm run dev
```

Useful scripts: `npm run build`, `npm start`, `npm run lint`, `npm run typecheck`, `npm run format`.

10.2 Production (typical: Vercel)

1. Set all production env vars (including `NEXT_PUBLIC_APP_URL` = production origin).
2. Configure Vercel Blob (or equivalent) so OneDrive tokens survive serverless instances.
3. Register production redirect URIs in Entra.
4. Connect ACS email domain + set sender address.
5. Sign in at `/setup`, connect OneDrive, verify Settings and a test link.

10.3 Smoke-test checklist

- ☐ Admin can sign in at `/setup`
 - ☐ Receiving OneDrive shows connected
 - ☐ Child names load from Excel
 - ☐ Generate link → open on phone Safari → upload small video
 - ☐ File appears in OneDrive with expected name
 - ☐ Notification email arrives with working link
 - ☐ Same link cannot upload again
-

11. Maintainability guide

11.1 Important files

Path	Responsibility
lib/server.ts	Central server logic (large — prefer extracting cohesive modules when changing)
lib/appConfig.ts	Shared types and defaults
lib/upload.ts	Browser upload orchestration
lib/email.ts	ACS send helper
lib/uploadFilename.ts / lib/age.ts	Naming and age math
app/fileprovider.tsx	Client upload state + context fetch
app/page.tsx	Parent UI (desktop + mobile layouts)
components/uploadArea.tsx	File picker / upload controls
components/settingsCard.tsx	Admin settings form
app/setup/page.tsx	Admin console
AGENTS.md	Next.js 16 note: read node_modules/next/dist/docs/ before framework changes

11.2 Conventions

- App Router only; "server-only" on sensitive server modules.
- "use client" only where interactivity requires it.
- Path alias @/ for imports.
- Prefer fixing auth/upload issues at the route + cookie layer rather than adding middleware unless needed.
- Keep mobile upload on native file inputs; avoid programmatic `.click()` / `dropzone open()` on iOS.

11.3 Known hardening / design notes

1. **GET / DELETE /api/links** — consider requiring admin auth before public production traffic.
2. **GET /api/config** — public; exposes limits and allowlisted admin emails.
3. **lib/server.ts size** — split Graph, Redis links, and OAuth when making large changes.
4. **Default admin email in code defaults** — prefer env/Settings as source of truth.
5. **Portal required for naming** — uploads without a portal link token cannot resolve the GMA filename.
6. **Email is best-effort** — monitor server logs for ACS errors (`DomainNotLinked` , auth failures).
7. **Secrets** — never commit `.env.local` ; rotate ACS keys and Azure client secrets if exposed.

11.4 Extending the system safely

Change	Suggested approach
New setting	Add to AppConfig + defaults + Settings UI + updateAppConfig merge
New API	Place under app/api/.../route.ts; call <code>canAccessUploadPortal</code> or <code>hasValidAdminAccess</code> explicitly
Filename scheme	Adjust lib/uploadFilename.ts + keep age logic in lib/age.ts
Extra notification channel	Hook beside <code>sendUploadNotificationEmail</code> in /api/upload/complete
UI copy	Prefer app/page.tsx / setup page; keep legal copy via lib/publisher.ts

11.5 Troubleshooting quick reference

Symptom	Likely cause
Upload button stays disabled	Missing date recorded, or <code>/api/upload/context</code> missing name/EDC (open via portal link)
Works in desktop DevTools, fails on iPhone	Use real device testing; ensure mobile file input path; empty MIME handling
Graph / SPO license errors	Receiving account lacks OneDrive/SharePoint capability
DomainNotLinked email error	Sender domain not connected to the ACS resource used by the connection string
OneDrive reconnect keeps old account	Clear token cache path / Blob; use Change receiving OneDrive flow
HMR blocked from phone IP	Add host to <code>allowedDevOrigins</code> in <code>next.config.ts</code>

12. Document control

Item	Value
Title	DDAGMA Upload Portal — System Documentation
Formats	Markdown source, DOCX, PDF under <code>/docs</code>
Scope	API routing, file handling, operator instructions, maintainability
Out of scope	Full Entra / ACS Azure portal screenshots (see Microsoft docs for UI steps)

For questions about publisher identity or legal pages, see `lib/publisher.ts` and the `/info`, `/privacy`, and `/tos` routes.